However, it's important to remember that an edge server adds complexity to the system. It's another component to build, deploy, and manage in production. It also adds a new network hop to the system, so the response time will increase. That's usually an insignificant cost, but you should keep it in mind. Since the edge server is the entry point to the system, it's at risk of becoming a single point of failure. As a basic mitigation strategy, you should deploy at least two replicas of an edge server following the same approach we discussed for configuration servers in chapter 4.

Spring Cloud Gateway greatly simplifies building edge services, focusing on simplicity and productivity. Furthermore, since it's based on a reactive stack, it can scale efficiently to handle the high workload naturally happening at the edge of a system.

The following section will teach you how to set up an edge server with Spring Cloud Gateway. You'll learn about routes, predicates, and filters, which are the building blocks of the gateway. And you'll apply the retry and timeout patterns you learned in the previous chapter to the interactions between the gateway and the downstream services.

NOTE If you haven't followed along with the examples implemented in the previous chapters, you can refer to the repository accompanying the book and use the project in Chapter09/09-begin as a starting point (https://github.com/ThomasVitale/cloud-native-spring-in-action).

9.1.1 Bootstrapping an edge server with Spring Cloud Gateway

The Polar Bookshop system needs an edge server to route traffic to the internal APIs and address several cross-cutting concerns. You can initialize our new Edge Service project from Spring Initializr (https://start.spring.io),